Q: Any questions or comments?

Lecture 19: Data & Failure

COMP2014 Distributed Systems

Chris Greenhalgh

School of Computer Science

Contents

- 2-phase commit
- Data replication
  - Passive replication
  - (Active replication)
  - Network partition

Note, these are also examples of both distributed algorithms and fault tolerance

Example: 2-phase Commit (section)

A distributed algorithm addressing partial failure when using multiple databases

2-Phase commit

- Consider a **distributed** banking **transaction**…
  - Several linked operations on different bank accounts managed by different banks / servers
    - E.g. a group of related withdrawals and deposits

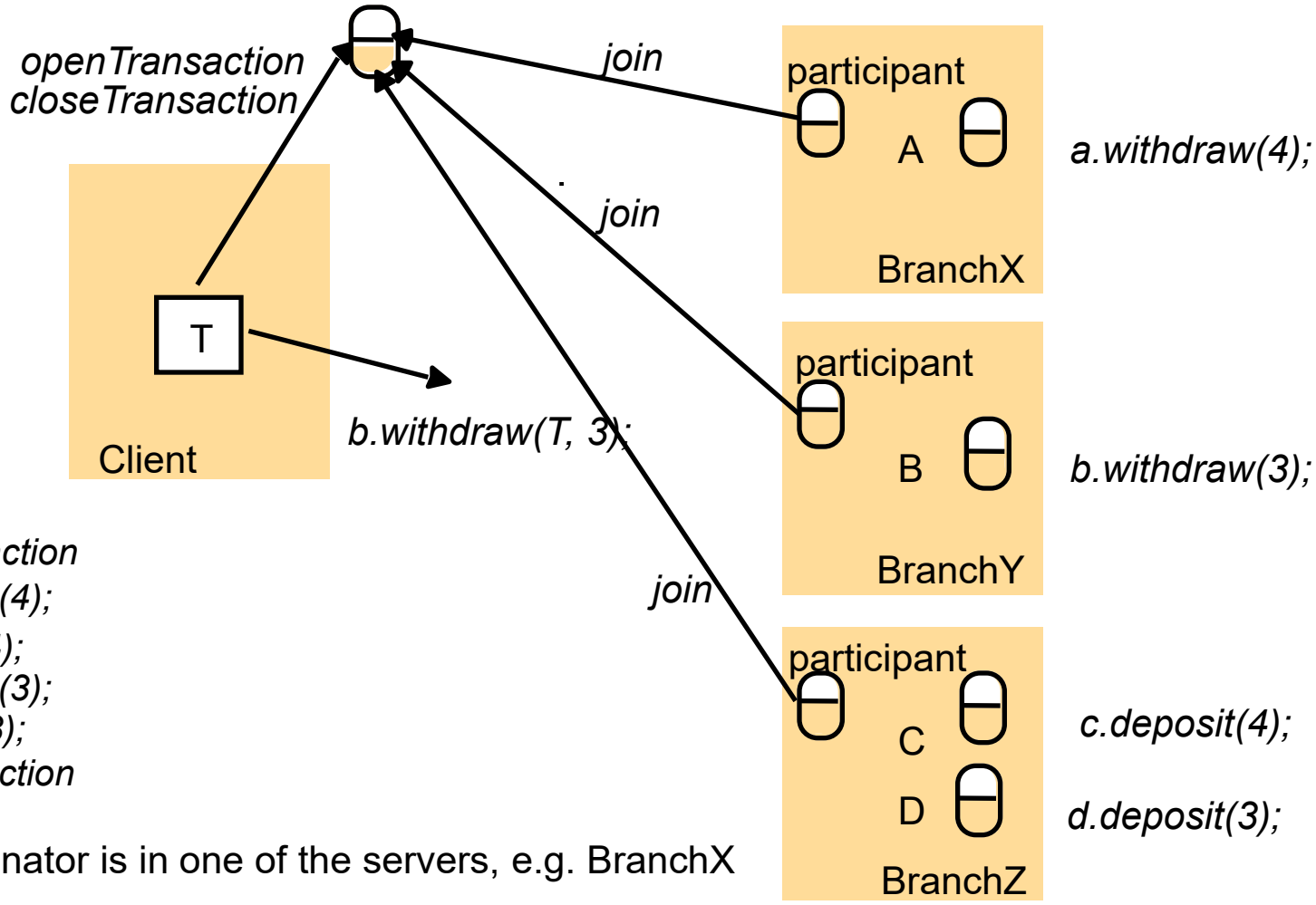
```
a.withdraw(4);
c.deposit(4);
b.withdraw(3);
d.deposit(3);
```

  - Any action might **fail**
    - E.g. an account has insufficient credit for a withdrawal
- How can we ensure that either *all* operations succeed or *no* operations occur?

Approach

- We'll assume that processes can be **trusted**, there are no arbitrary failures and communication is **reliable**
  - and (for now) that processes don't crash
- Set aside a specialised **coordinator** process to **coordinate** the entire transaction
  - Each **participant** process **joins** the transaction

A distributed banking transaction



Note: the coordinator is in one of the servers, e.g. BranchX

Algorithm overview

- Each participant breaks down the operation into two **stages** or **phases**:
  - **Tentatively** performing the operation
    - If this specific operation is going to **fail** then the process will discover at this point
  - Making the successful result permanent and visible, i.e. **committing** the result
    - Alternatively it can **abort** the operation, discarding any tentative changes
- These two stages or phases are managed by the coordinator process:

The two-phase commit protocol

Phase 1 (voting phase):

1. The coordinator sends a *canCommit*? request to each of the participants in the transaction.
2. When a participant receives a *canCommit*? request it replies with its vote (*Yes* or *No*) to the coordinator. Before voting *Yes*, it prepares to commit by saving objects in permanent storage. If the vote is *No* the participant aborts immediately.

Phase 2 (completion according to outcome of vote):

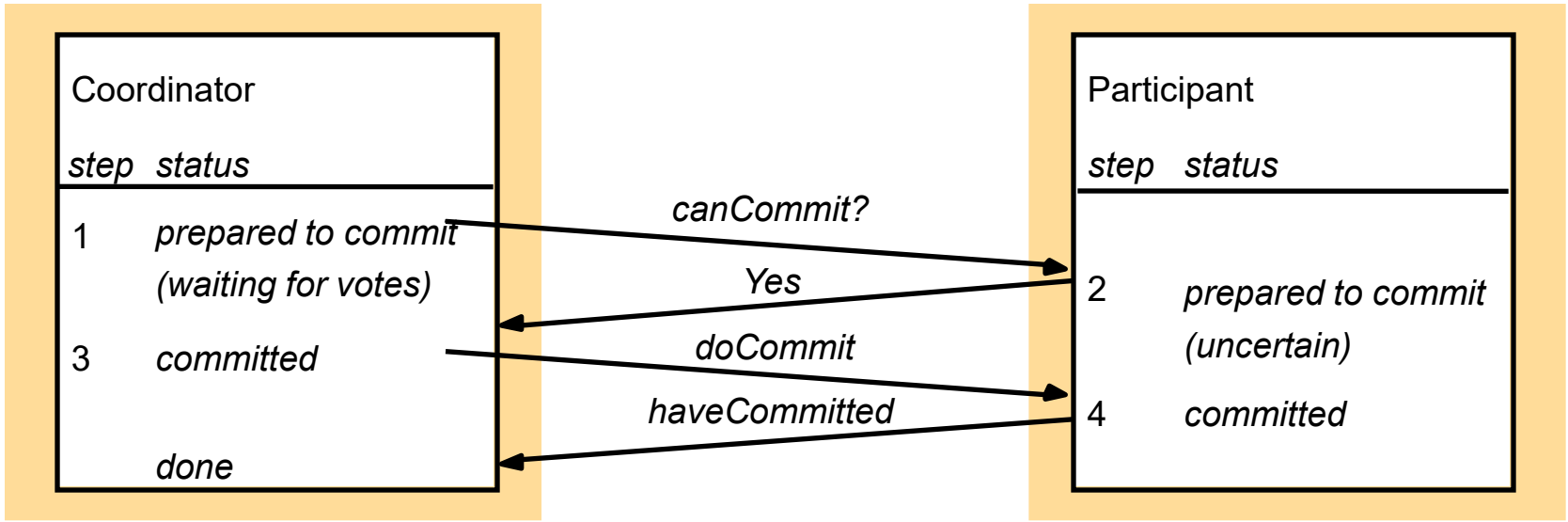
3. The coordinator collects the votes (including its own).

(a) If there are no failures and all the votes are *Yes* the coordinator decides to commit the transaction and sends a *doCommit* request to each of the participants.

(b) Otherwise the coordinator decides to abort the transaction and sends *doAbort* requests to all participants that voted *Yes*.

4. Participants that voted *Yes* are waiting for a *doCommit* or *doAbort* request from the coordinator. When a participant receives one of these messages it acts accordingly and in the case of commit, makes a *haveCommitted* call as confirmation to the coordinator.

Communication in two-phase commit protocol



Notes
(2-phase commit)

- So if **no** operation fails then all processes will reply to *canCommit?* with *Yes*
  - And **all** of the operations will be **committed**
  - So the effects of **all operations** will be observed
- So if **any** operation fails then that process will reply to *canCommit?* with *No*
  - And **all** of the operations will be **aborted**
  - So **no** effect of any **operation** will be observed

Failures…

Q: what does happen if a process fails?

- But what will happen if a process **crashes**??
- Depends **when**, e.g.
  - before any *doCommit* the transaction can be aborted,
  - but afterwards it cannot
  - We need to be sure in that case that the process will be restarted and will definitely commit that operation
- (I.e. more assumptions / constraints…)
  - (typically that it has recorded to persistent storage what it needs to do, and it will check with the transaction manager when it restarts before doing anything else)

Data Replication (Section)

Recall…

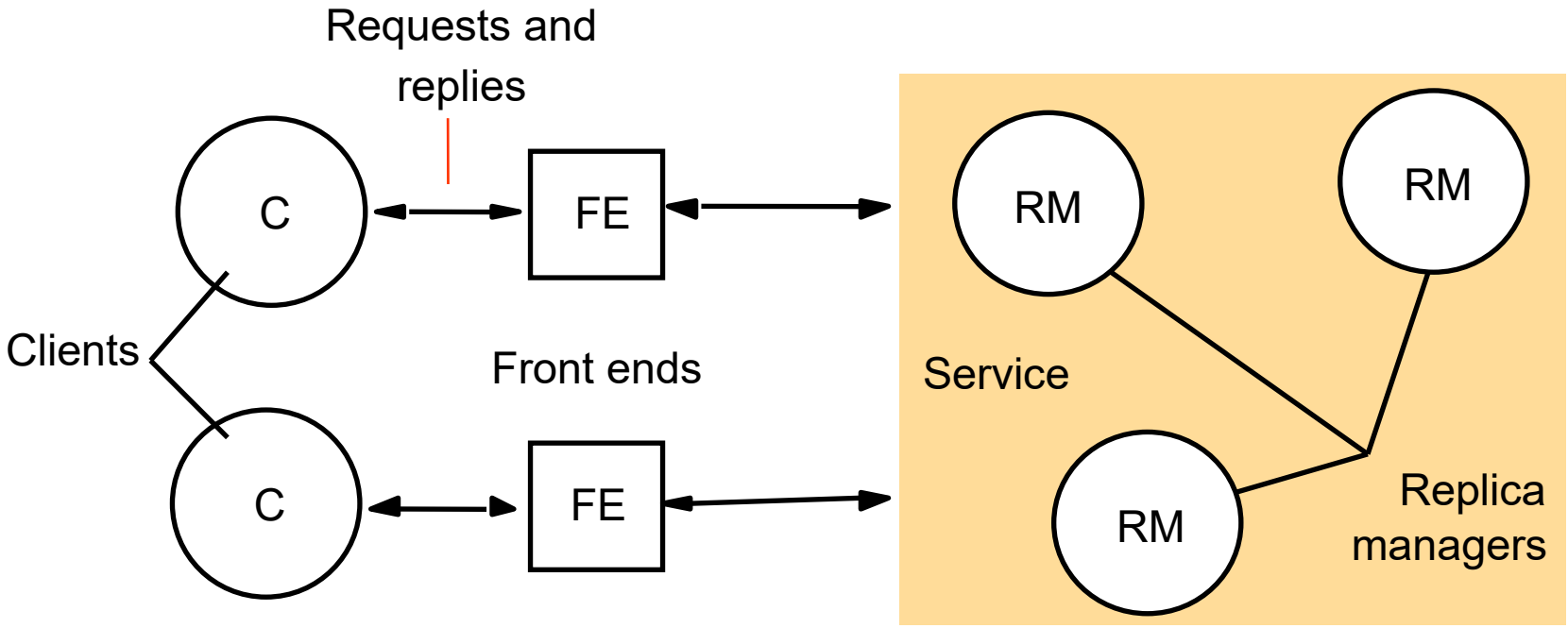
- **Replication** can be used to provide **fault tolerance, performance** and **availability**
- Usually **replication transparency** is desirable, but **consistency** requirements may depend on the application
- What about general replicated data services?
  - E.g. replicated databases

A system model for managing replicated data

(quite general, but not the only possible approach)

- We will assume **physical** replicas of **objects** are maintained by **Replica Managers** (RMs)
- A **service** is provided by a cooperating **set** of RMs
- Each **Client** (only) makes **requests** via a **Front End**
  - May be part of the client, or may be a separate process
  - Provides the basis for client **replication transparency**
    - Client does not interact with replicas directly
- Requests can be ***read-only*** or ***update*** (i.e. read/write)
  - Note: if all data is read-only (i.e. immutable) then replication is MUCH easier than if updates are allowed

A basic architectural model for the management of replicated data



Phases of request handling
(general framework)

1. *Request* issued by FE
   - **Either to one RM**
     or **multicast to all RMs**
2. *Coordination* between RMs on whether to perform request and in what order
   - Typically FIFO, or may be Causal or Total order – see group communication (lecture 12)
3. *Execution* by RMs
   - possibly tentative, e.g. in a transaction
4. *Agreement* by RMs on effect (if any)
5. *Response* by one or more RMs to FE

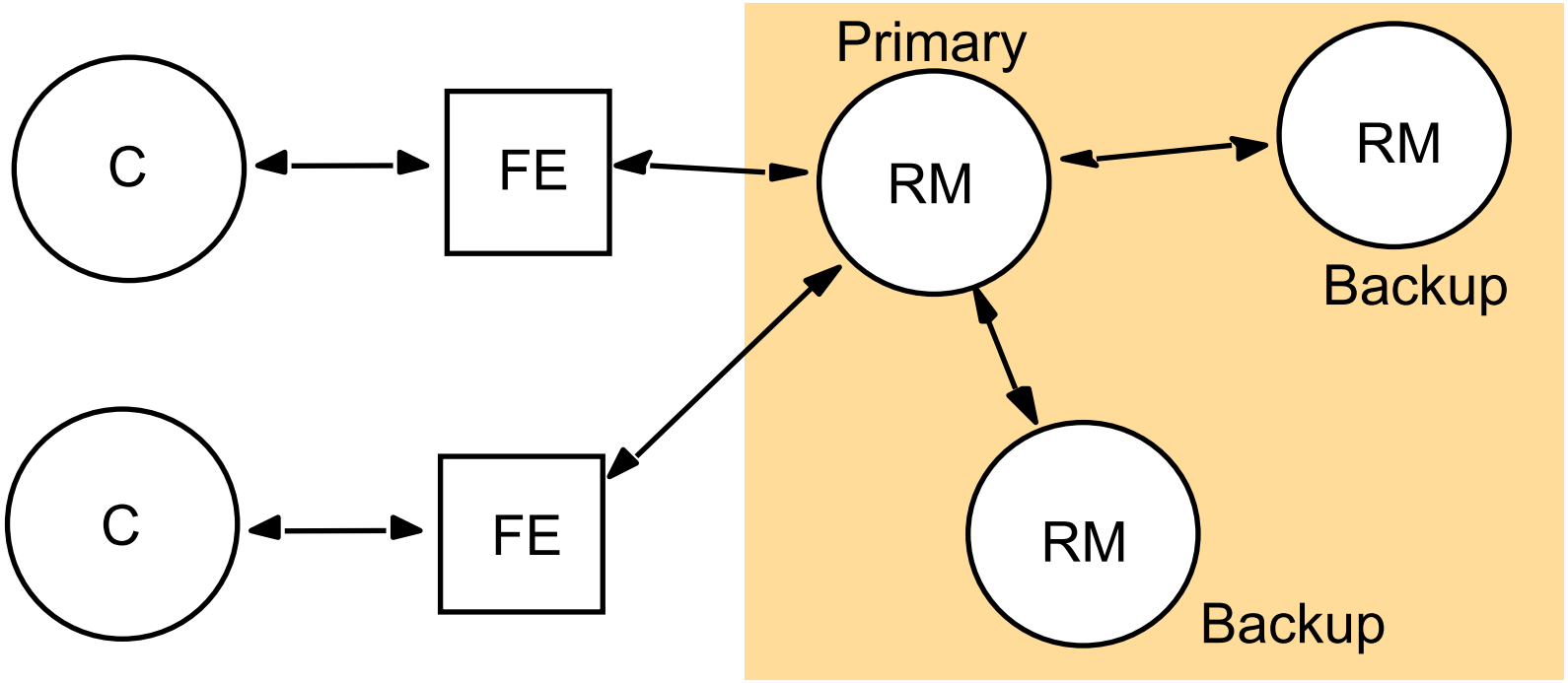
Fault tolerant service

- We want to use this model to design a **fault tolerant** service which can tolerate (say) up to ***f*** **process failures**
  - Need to specify **crash** failures or **Byzantine** failures
    - (Byzantine failures much harder to deal with)
- (intuitively) **correct** if each client gets the same results as they would have got from a single correct Replica Manager
  - But note that operations from different clients might be **reordered** with respect to one another compared to a single server system
    - E.g. with per-client FIFO ordering

Option: Passive (primary backup) replication request handling

1. *Request*: (with unique ID) is sent by the client's Front End to one "**primary**" Replica Manager
2. *Coordination*: the primary takes requests in order
   - The other RMs are not involved
   - If request has already been handled just resend the response
3. *Execution*: the primary executes & stores result
4. *Agreement*: if request is an update, the primary sends updated state, response and ID to all **backup** RMs
   - Backup RMs confirm that they have made the update
5. *Response*: primary responds to FE and hence client

The passive (primary-backup) model for fault tolerance



How does it achieve fault tolerance?

Q: what does happen if a process fails?

- For correct operation, if the **primary fails** then
  - The backups must agree on one backup to be the **new primary**
  - All RMs must agree on which **operations** had been performed at the point when new primary takes over
- If a **backup fails** then the rest just continue
- If a failed RM **recovers** then it must re-synchronise
  - i.e. ensure that it has the same state/responses as the current primary

Notes (passive replication)

- Surviving up to *f* crashes requires at least *f*+1 Replica Managers
- Cannot cope with Byzantine failures
  - E.g. primary RM sending incorrect updates
- Has relatively large **overheads**
  - i.e. all the backup RMs which are not doing much most of the time
- Client *may* be able to submit read requests (not updates) to backup RMs
  - => Increased **performance** but reduced **consistency**, because a backup may not have the latest update yet

Option 2: Active replication

Details not examinable

1. *Request*: (with unique ID) is sent by Front End to ***all*** Replica Managers
   - E.g. using reliable totally ordered **multicast**
2. *Coordination*: the multicast group delivers the request to all active RMs
3. *Execution*: every RM executes the request
4. *Agreement*: none required with reliable multicast
   - At least provided that the multicast is reliable, view-consistent and totally ordered and operations are atomic and deterministic!
5. *Response*: every RM sends response to FE
   - FE may take **first** response, or (e.g.) wait for **majority** depending on failure assumptions

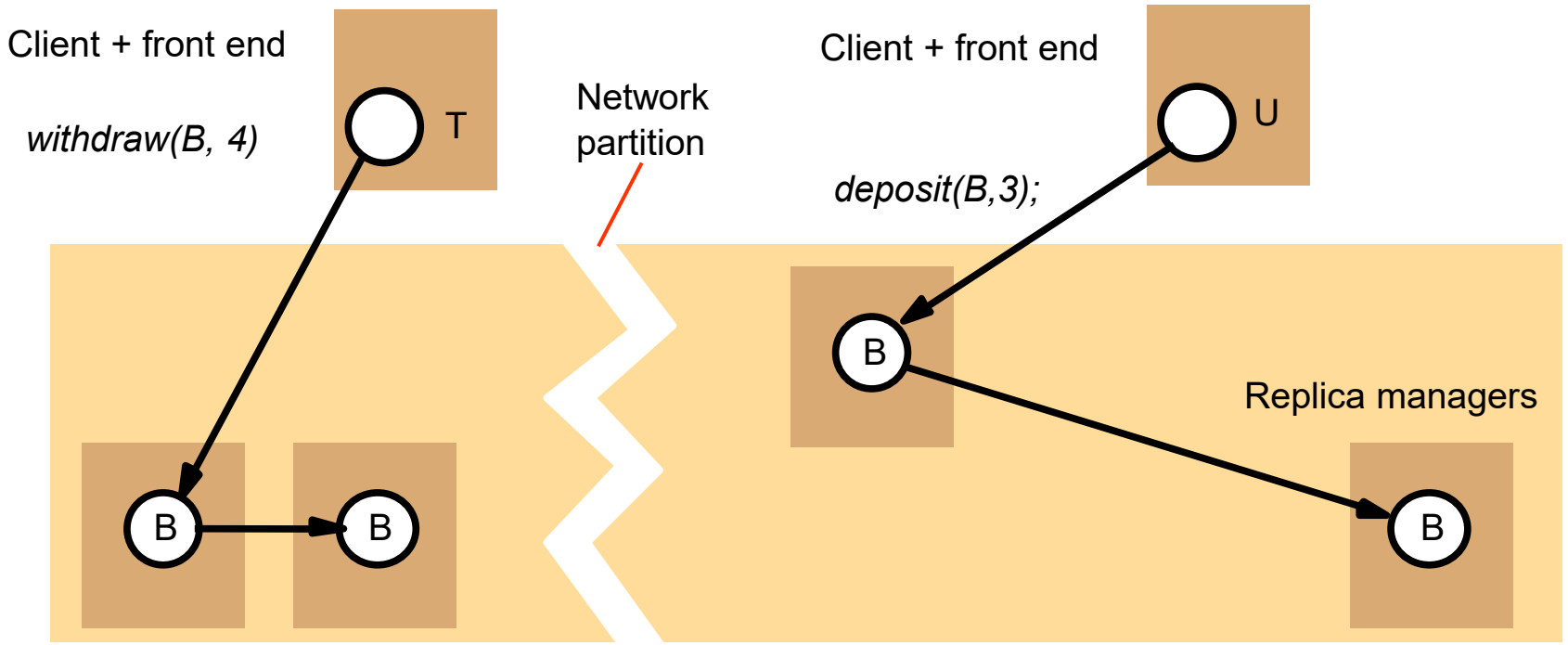
Notes (active replication)

- For **crash** failures requires f+1 replicas to tolerate f failures
- Can cope with **Byzantine** (i.e. arbitrary) failures
  - Requires at least 2f+1 replicas to tolerate up to f Byzantine failures
    - Each FE collects f+1 identical responses before replying to client
- Weaker ordering or participation of a subset of RMs may be OK in some applications
  - Will give reduced **consistency**

Partitions

- But what happens for example if the network is **partitioned**
  - => updates in one partition cannot reach the other partition
  - => cannot distinguish partition from the crash failure of the replicas “beyond” the partition…

Network partition



Instructor's Guide for Coulouris, Dollimore, Kindberg and Blair, Distributed Systems: Concepts and Design Edn. 5
© Pearson Education 2012

Should the operations be allowed or not??

Some options for dealing with partition

- *All copies*
  - Require **ALL** RMs to participate in every request
    - => will not work if *any* replica fails or any partition
- *Available copies*
  - Use whichever replicas are currently **available**
    - => Will not work correctly with partition, as available copies in each partition handle different updates and may make incompatible changes
- *Quorum consensus*:
  - Require a **quorum** (minimum set) of replicas to perform an operation, typically a majority of replicas
    - i.e. at most one partition – with the majority of processes – can still perform operations
    - => lower availability but high consistency

2-Phase Commit Summary

- (For example) **2-phase commit** can realise **distributed transactions** in the presence of partial failure
  - => Either all actions succeed, or no change is made
- Note, as a distributed algorithm it:
  - Has specific goals/success criteria
  - Defines assumptions, i.e. the kind(s) of failure that are considered
  - Identifies specific roles for processes
  - Sets out the steps to be taken by *each process* in the distributed system
  - Including the messages sent/received (=protocol)

Data Replication Summary

- A replicated **data service** may be provided by a cooperating **set** of **Replica Managers** (RMs)
  - which manage **physical** replicas of **logical** objects
  - Each **Client** makes **requests** in terms of logical objects via a **Front End** which provides replication transparency
- Can provides **fault tolerance**
  - e.g. using passive (primary backup) replication
- Depending on the specific approach used this can operate correctly with network **partition**
  - by requiring a suitable quorum of RMs for each operation

Next

- Drop-in / catch-up lab(s)
- Revision / Q&A lectures…
- Assessment ☺